# Prefix Sum(Scan) and Extensions to CUDA

CSCD439/539 GPU Programming

# Outline

- Prefix Sum(Chapter 9)
- Prefix Sum with Thrust

# Prefix-sum(chapter 9)

- Parallel prefix-sums, **or scan** operations, are important building blocks (**parallel pattern**) in many parallel algorithms,
  - such as stream compaction(SC) and radix sort.
  - SC removes unwanted elements in sparse data

# Prefix-sum

- **exclusive scan**,
  - each element **j** of the result is the sum of all elements up to but not including j in the input array.
- **inclusive scan**
  - all elements including j are summed.
- An exclusive scan can be generated from an inclusive scan by shifting the resulting array right by one element and inserting the identity.

# Prefix-sum

- **exclusive scan**
- Input: [3 1 7 0 4 1 6 3]
- Output: [0 3 4 11 11 15 16 22]
- **inclusive scan**
- Input : {1, 0, 2, 2, 1, 3}
- Output: {1, 1, 3, 5, 6, 9}

# Prefix-sum

- **Two phases in Parallel programming**

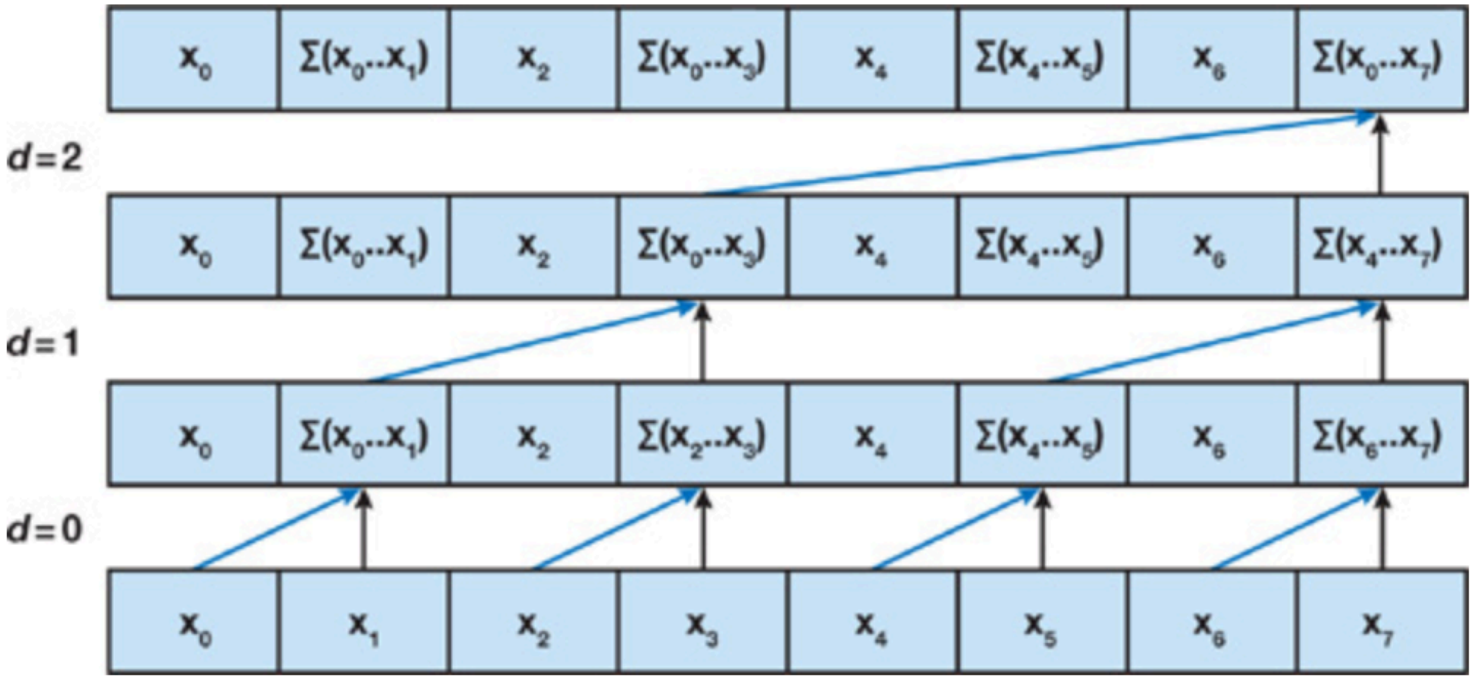



Figure 39-3 An Illustration of the Up-Sweep, or Reduce, Phase of a Work-Efficient Sum Scan Algorithm

# Prefix-sum

- **Two phases in Parallel programming**

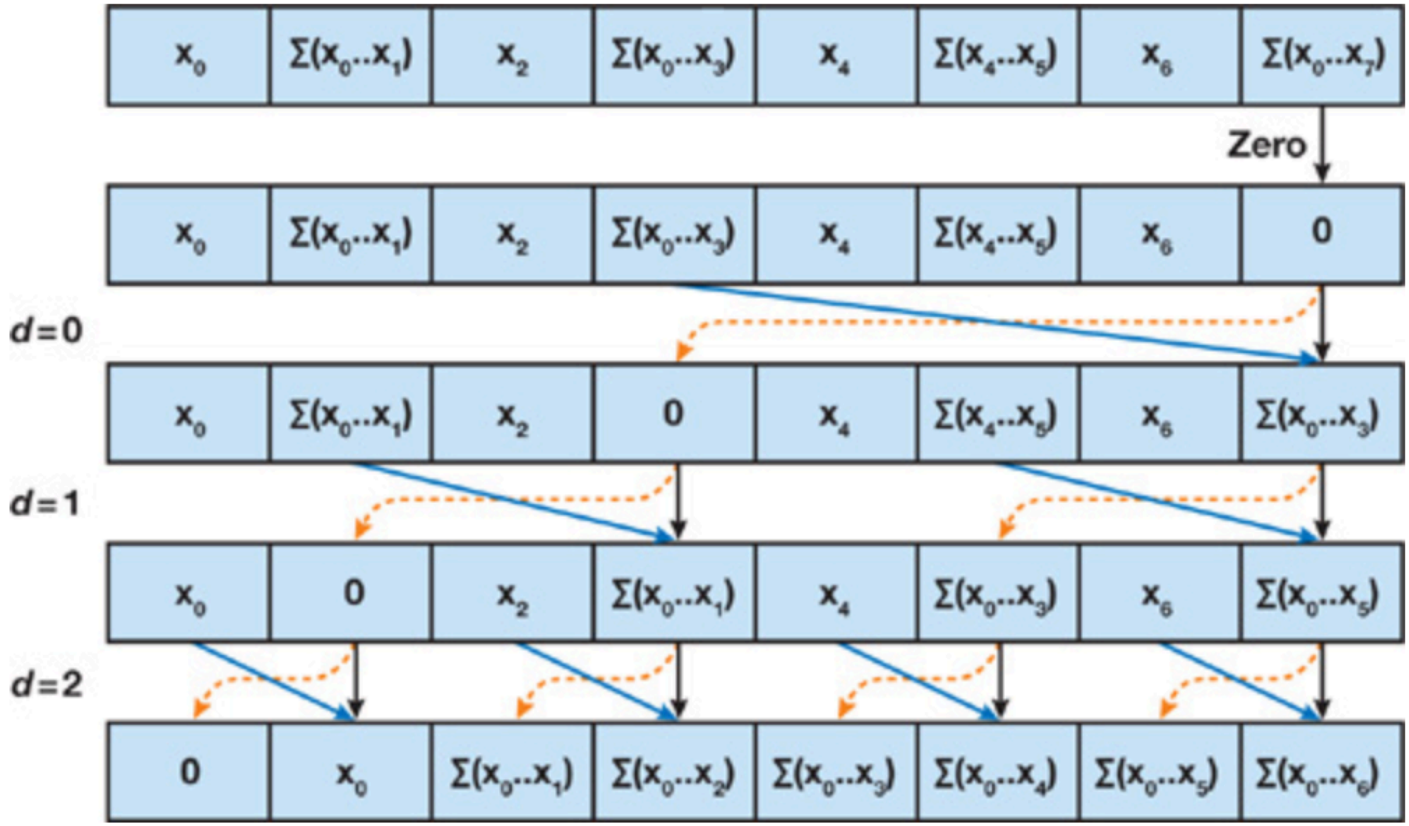



Figure 39-4 An Illustration of the Down-Sweep Phase of the Work-Efficient Parallel Sum Scan Algorithm

# Prefix-sum with Thrust

```
#include <thrust/scan.h>

int data[6] = {1, 0, 2, 2, 1, 3};

thrust::inclusive_scan(data, data + 6, data); // in-place scan

// data is now {1, 1, 3, 5, 6, 9}
//For example, data[2] = data[0] + data[1] + data[2].
```

An exclusive scan is similar, but shifted by one place to the right:

# Prefix-sum with Thrust

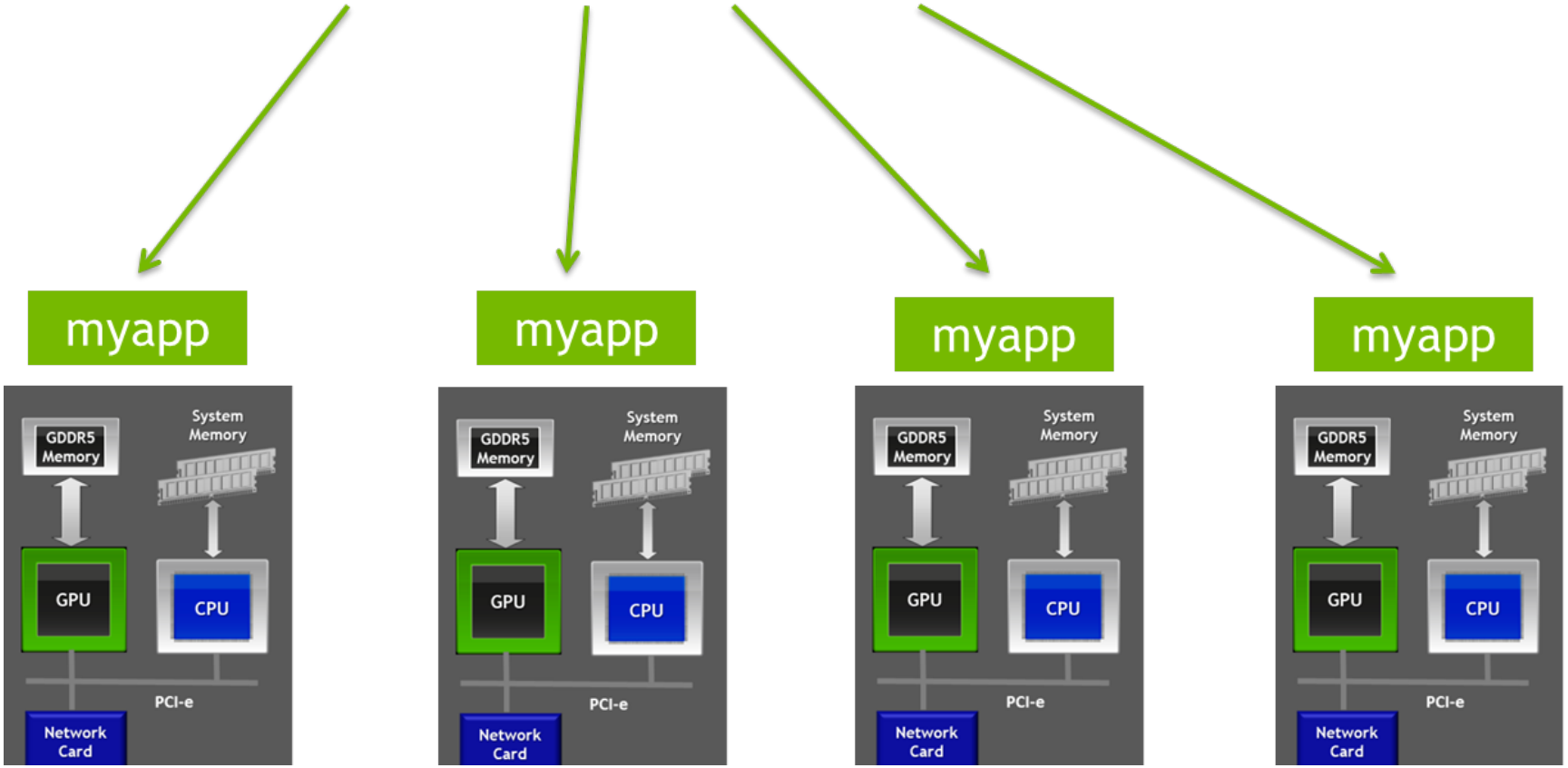
```
#include <thrust/scan.h>

int data[6] = {1, 0, 2, 2, 1, 3};

thrust::exclusive_scan(data, data + 6,
data); // in-place scan
// data is now {0, 1, 1, 3, 5, 6}
// So now data[2] = data[0] + data[1]
```

# Cuda-MPI joint Programming

```
mpirun -np 4 ./myapp <args>
```



# Cuda-MPI joint Programming

- One company making GPUs and NIC(network interface controller) combined on a single chip(board).
- Kepler GK110 supports the RDMA feature in NVIDIA GPUDirect, which is designed to improve performance by allowing direct access to GPU memory by third-party devices such as IB adapters, NICs, and SSDs.

# Cuda-MPI joint Programming

- When using CUDA 5.0, GPUDirect provides the following important features:
  - Direct memory access (DMA) between NIC and GPU without the need for CPU-side data buffering.
  - Significantly improved MPISend/MPIRecv efficiency between GPU and other nodes in a network.
  - Eliminates CPU bandwidth and latency bottlenecks
  - Works with variety of 3rd-party network, capture, and storage devices

# Dynamic Parallelism(Chapter 20)

- CUDA dynamic parallelism is an extension to the CUDA programming model enabling a CUDA kernel to create new thread grids by launching new kernels.
- Dynamic parallelism is introduced with the Kepler architecture, first appearing in the GK110 chip.

# Dynamic Parallelism

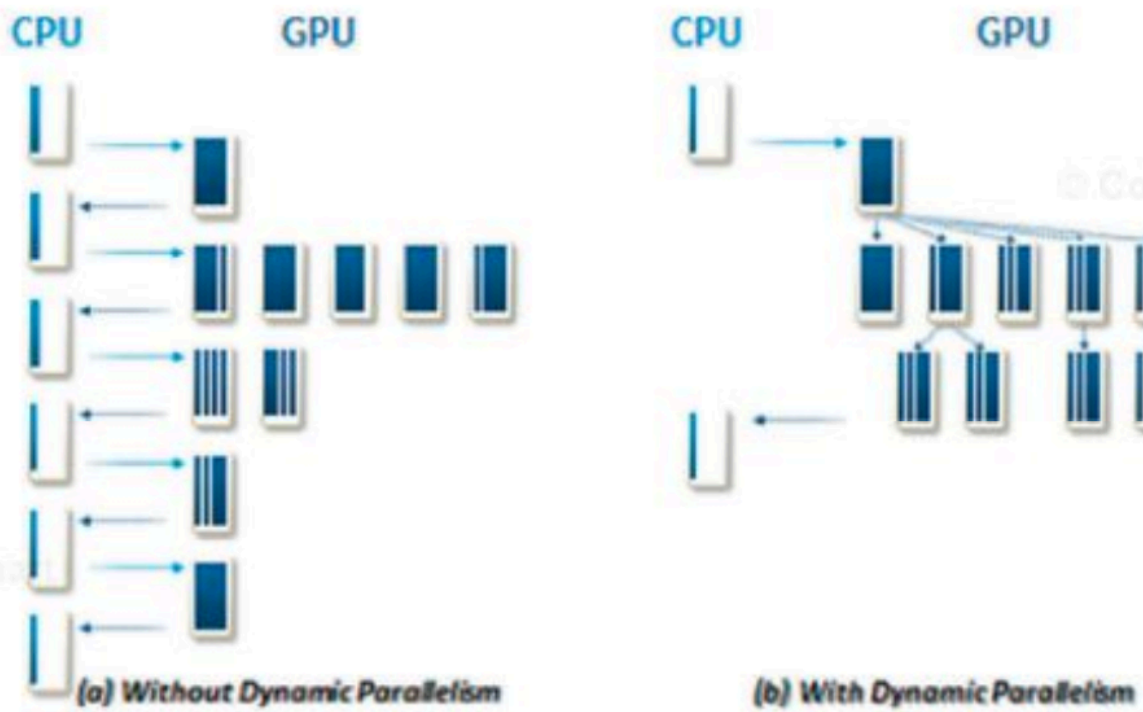



**FIGURE 20.2**

Kernel launch patterns for algorithms with dynamic work variation: (a) without dynamic parallelism and (b) with dynamic parallelism.

# Summary

- Parallel Prefix-Sum, aka, scan
- Cuda-MPI joint programming
  - Use GPUs in distributed systems, cloud, grid computing.
  - Distributed big data processing
- Dynamic Parallelism

# Summary

- If you like to do a project using these libraries,
  - OpenACC
  - Thrust
  - Cuda-MPI
- Data mining, Database on GPU, big data, Distributed systems(cloud comuting), text mining, image processing, graphics, gaming etc.
- Let us work together!!

# Summary

- If you feel you learned a lot in this class,
- Please tell your friends and ask them to take this class.
- But, also tell them it is challenging!!